

# Custom Notification Templates

**Series:** WFLOW | **Notebook:** 6 of 9 | **Created:** January 2026 | **Last Updated:** 01/28/2026

## Rich Message Formatting

Create professional, informative notifications with dynamic content, formatting, and data enrichment. This notebook covers Jinja templating, Slack Block Kit, Teams Adaptive Cards, and data enrichment patterns.

## Table of Contents

- 1. [Template Design Principles](#)
- 2. [Jinja2 Expression Deep Dive](#)
- 3. [Slack Block Kit Templates](#)
- 4. [Teams Adaptive Card Templates](#)
- 5. [Data Enrichment](#)
- 6. [Template Library](#)
- 7. [Testing Templates](#)

## Prerequisites

| Requirement           | Details                                 |
|-----------------------|-----------------------------------------|
| Dynatrace Environment | SaaS with Platform subscription         |
| Permissions           | <code>automation:workflows:write</code> |

| Requirement     | Details                   |
|-----------------|---------------------------|
| Prior Knowledge | WFLOW-01 through WFLOW-05 |

# 1. Template Design Principles

## Effective Alert Messages

| Principle      | Good                      | Bad                        |
|----------------|---------------------------|----------------------------|
| Scannable      | Key info in first line    | Important details buried   |
| Actionable     | Include link to problem   | "Check Dynatrace"          |
| Contextual     | Show affected service     | Generic "problem detected" |
| Severity-coded | Visual severity indicator | Plain text only            |
| Concise        | Essential details only    | Long descriptions          |

## Message Structure

```
[1] SEVERITY INDICATOR + TITLE
[2] Key metrics/impact
[3] Affected entities
[4] Action button/link
```

## Severity Visual Guide

| Severity | Slack                 | Teams         | Email         |
|----------|-----------------------|---------------|---------------|
| CRITICAL | :red_circle:          | Red header    | Red banner    |
| HIGH     | :large_orange_circle: | Orange header | Orange banner |
| MEDIUM   | :large_yellow_circle: | Yellow header | Yellow banner |
| LOW      | :large_blue_circle:   | Blue header   | Blue banner   |

## 2. Jinja2 Expression Deep Dive

---

### Variable Access

```
{{ event()["title"] }}           {# Direct field access #}
{{ event().get("field", "default") }} {# With default value #}
{{ result("task_name").output }} {# Previous task result #}
{{ env.SECRET_NAME }}           {# Environment secret #}
{{ now() }}                     {# Current timestamp #}
```

### Filters

```
{{ event()["title"] | upper }}   {# UPPERCASE #}
{{ event()["title"] | lower }}   {# lowercase #}
{{ event()["title"] | truncate(50) }} {# Limit length #}
{{ list_field | join(", ") }}   {# Join array #}
{{ number | round(2) }}         {# Round decimals #}
{{ timestamp | format_datetime }} {# Format date #}
```

### Conditionals

```
{% if event()["severity"] == "CRITICAL" %}
    :rotating_light: CRITICAL ALERT
{% elif event()["severity"] == "HIGH" %}
    :warning: HIGH ALERT
{% else %}
    :information_source: {{ event()["severity"] }} ALERT
{% endif %}
```

### Loops

```
Affected Entities:
{% for entity in event()["affected_entity_ids"][:5] %}
    - {{ entity }}
{% endfor %}
{% if event()["affected_entity_ids"] | length > 5 %}
    ... and {{ event()["affected_entity_ids"] | length - 5 }} more
{% endif %}
```

## Inline Conditionals

```
{{ ":red_circle:" if event()["severity"] == "CRITICAL" else  
":large_yellow_circle:" }}
```

  

```
{{ event().get("root_cause_entity_id", "Pending analysis") }}
```

## Dictionary Mapping

```
{# Severity to emoji mapping #}  
{{ {  
    "CRITICAL": ":red_circle:",  
    "HIGH": ":large_orange_circle:",  
    "MEDIUM": ":large_yellow_circle:",  
    "LOW": ":large_blue_circle:"  
}.get(event()["severity"], ":white_circle:") }}
```

### 3. Slack Block Kit Templates

---

#### Full-Featured Alert Template

```
input:
  connection: slack-production
  channel: "#alerts-production"
  blocks:
    # Header with severity
    - type: header
      text:
        type: plain_text
        text: "{{ {\\"CRITICAL\\": \":rotating_light:\", \\"HIGH\\":
\\:warning:\", \\"MEDIUM\\": \":large_yellow_circle:\", \\"LOW\\":
\\:information_source:\"}.get(event()[\\"severity\\"], \":grey_question:\") }}"
        {{ event()[\\"severity\\"] }} Problem"

    # Problem title
    - type: section
      text:
        type: mrkdown
        text: "*{{ event()['title'] }}"

    # Details in two columns
    - type: section
      fields:
        - type: mrkdown
          text: "*Problem ID:*\\n{{ event()['display_id'] }}"
        - type: mrkdown
          text: "*Status:*\\n{{ event()['status'] }}"
        - type: mrkdown
          text: "*Started:*\\n{{ event()['start_time'] }}"
        - type: mrkdown
          text: "*Severity:*\\n{{ event()['severity'] }}"

    # Root cause (if available)
    - type: section
      text:
        type: mrkdown
        text: "*Root Cause:*\\n{{ event().get('root_cause_entity_id',
'Analysis in progress...') }}"

    # Affected entities
    - type: section
      text:
```

```
    type: mrkdown
    text: "*Affected ({{ event()['affected_entity_ids'] | length
}}):*\n{{ event()['affected_entity_ids'][:3] | join('\n') }}{{ '\n...' if
event()['affected_entity_ids'] | length > 3 else '' }}"

# Divider
- type: divider

# Action buttons
- type: actions
  elements:
    - type: button
      text:
        type: plain_text
        text: "View Problem"
      url: "{{ event()['problem_url'] }}"
      style: primary
    - type: button
      text:
        type: plain_text
        text: "View Service"
      url: "https://{{ env.DYNATRACE_HOST }}/ui/services"

# Footer context
- type: context
  elements:
    - type: mrkdown
      text: "Detected by Dynatrace Dynatrace Intelligence | {{ now() }}"
```

## 4. Teams Adaptive Card Templates

---

### Full-Featured Alert Card

```
input:
  connection: teams-production
  card:
    type: AdaptiveCard
    $schema: "http://adaptivecards.io/schemas/adaptive-card.json"
    version: "1.4"
    body:
      # Header with color
      - type: TextBlock
        text: "{{ event()['severity'] }}" Problem Detected"
        size: Large
        weight: Bolder
        color: "{{ {'CRITICAL': 'Attention', 'HIGH': 'Warning', 'MEDIUM':
'Accent', 'LOW': 'Good'}.get(event()['severity'], 'Default') }}"

      # Problem title
      - type: TextBlock
        text: "{{ event()['title'] }}"
        wrap: true
        weight: Bolder

      # Details table
      - type: FactSet
        facts:
          - title: "Problem ID"
            value: "{{ event()['display_id'] }}"
          - title: "Severity"
            value: "{{ event()['severity'] }}"
          - title: "Status"
            value: "{{ event()['status'] }}"
          - title: "Started"
            value: "{{ event()['start_time'] }}"
          - title: "Root Cause"
            value: "{{ event().get('root_cause_entity_id', 'Analyzing...')
}}"

      # Affected entities
      - type: TextBlock
        text: "Affected Entities ({{ event()['affected_entity_ids'] | length
}})"
        weight: Bolder
```

```
        spacing: Medium
      - type: TextBlock
        text: "{{ event()['affected_entity_ids'][:5] | join(', ') }}{{ ' ',
... ' if event()['affected_entity_ids'] | length > 5 else ' ' }}"
        wrap: true
        size: Small

    actions:
      - type: Action.OpenUrl
        title: "View in Dynatrace"
        url: "{{ event()['problem_url'] }}"
```

## 5. Data Enrichment

---

### Enrich with DQL Query

Add recent error logs to notification:



```

import { queryExecutionClient } from '@dynatrace-sdk/client-query';

export default async function({ event }) {
  // Get recent error logs for the affected service
  const rootCause = event.root_cause_entity_id;

  if (!rootCause) {
    return { event, recent_errors: [] };
  }

  const result = await queryExecutionClient.queryExecute({
    body: {
      query: `
        fetch logs, from: now() - 30m
        | filter dt.entity.service == "${rootCause}"
        | filter loglevel == "ERROR"
        | fields timestamp, content
        | sort timestamp desc
        | limit 5
      `,
      requestTimeoutMilliseconds: 30000
    }
  });

  return {
    event,
    recent_errors: result.result.records || []
  };
}

```

## Use Enriched Data in Template

```

*Recent Error Logs:*
{% for log in result("enrich_data").recent_errors[:3] %}
  • `{{ log.content | truncate(100) }}`
{% endfor %}
{% if result("enrich_data").recent_errors | length == 0 %}
  _No recent errors found_
{% endif %}

```

## Enrich with Entity Details

```
import { entitiesClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function({ event }) {
  const entityId = event.root_cause_entity_id;

  if (!entityId) {
    return { entity_name: 'Unknown', entity_type: 'Unknown' };
  }

  const entity = await entitiesClient.getEntity({
    entityId: entityId
  });

  return {
    entity_name: entity.displayName,
    entity_type: entity.type,
    tags: entity.tags || []
  };
}
```

## 6. Template Library

---

### Compact Alert (Slack)

```
message: |
  {{ {"CRITICAL": ":red_circle:", "HIGH": ":large_orange_circle:", "MEDIUM":
  ":large_yellow_circle:", "LOW": ":large_blue_circle:"}.get(event()
  ["severity"], ":white_circle:") }} *{{ event()["title"] }}*
  `{{ event()["display_id"] }}` | {{ event()["severity"] }} | &lt;{{ event()
  ["problem_url"] }}|View&gt;
```

## Problem Resolved (Slack)

```
message: |
:white_check_mark: *Problem Resolved*

*{{ event()["title"] }}*

• *Duration:* {{ event().get("duration", "N/A") }}
• *Problem ID:* {{ event()["display_id"] }}
• *Resolved:* {{ event().get("end_time", now()) }}
```

## Daily Summary (Scheduled)

```
message: |
:bar_chart: *Daily Problem Summary*

*Last 24 Hours:*
• Problems opened: {{ result("query_stats").opened }}
• Problems resolved: {{ result("query_stats").resolved }}
• Currently open: {{ result("query_stats").open }}

*By Severity:*
:red_circle: Critical: {{ result("query_stats").critical }}
:large_orange_circle: High: {{ result("query_stats").high }}
:large_yellow_circle: Medium: {{ result("query_stats").medium }}
```

## Escalation Notice

```
message: |
:rotating_light: *ESCALATION*

Problem *{{ event()["display_id"] }}* has not been acknowledged after 15
minutes.

*{{ event()["title"] }}*

This alert is being escalated to the on-call team.
&lt;{{ event()["problem_url"] }}|View Problem&gt;
```

## 7. Testing Templates

---

### Test with On-Demand Trigger

1. Create workflow with On-Demand trigger
2. Add JavaScript task to create mock event:

```
export default async function() {
  return {
    mock_event: {
      display_id: "P-TEST-001",
      title: "High response time on checkout-service",
      severity: "CRITICAL",
      status: "OPEN",
      start_time: new Date().toISOString(),
      affected_entity_ids: ["SERVICE-ABC123", "SERVICE-DEF456"],
      root_cause_entity_id: "HOST-XYZ789",
      management_zones: ["Production", "Checkout"],
      problem_url: "https://example.dynatrace.com/ui/problems/P-TEST-001"
    }
  };
}
```

3. Use mock data in notification:

```
message: |
  {{ result("mock_data").mock_event.severity }} Alert
  {{ result("mock_data").mock_event.title }}
```

### Preview Templates in Slack Block Kit Builder

Use <https://app.slack.com/block-kit-builder> to preview your block templates before deployment.

## Query Notification Effectiveness

```
// Notification task performance
fetch events, from: now() - 7d
| filter event.type == "automation.task.execution"
| filter matchesPhrase(task.type, "slack") or matchesPhrase(task.type,
"msteams") or matchesPhrase(task.type, "email")
| summarize
    avg_duration = avg(task.duration),
    max_duration = max(task.duration),
    total = count(),
    by:{task.type}
| sort total desc
```

```
// Template errors (usually from malformed Jinja)
fetch events, from: now() - 24h
| filter event.type == "automation.task.execution"
| filter task.status == "FAILED"
| filter matchesPhrase(task.error, "template") or matchesPhrase(task.error,
"jinja") or matchesPhrase(task.error, "expression")
| fields timestamp, workflow.name, task.name, task.error
| sort timestamp desc
| limit 20
```

## Next Steps

---

With custom templates ready, implement auto-remediation:

### Recommended Path

1. **WFLOW-07: Problem-Triggered Remediation** - Auto-remediation patterns
2. **WFLOW-08: JavaScript & HTTP Actions** - Custom integrations
3. **WFLOW-09: Security & Governance** - Best practices

### Key Takeaways

- **Jinja2** enables dynamic, conditional content
- **Slack Block Kit** creates rich, interactive messages
- **Teams Adaptive Cards** provide structured formatting

- **Data enrichment** adds context from DQL queries
  - **Test templates** with mock data before production
- 

## Summary

---

In this notebook, you learned:

- Alert message design principles
  - Jinja2 expressions, filters, and conditionals
  - Slack Block Kit template structure
  - Teams Adaptive Card formatting
  - Data enrichment with DQL and entity lookups
  - Template library patterns
  - Testing strategies
- 

## References

---

- [Jinja2 Template Designer](#)
  - [Slack Block Kit](#)
  - [Teams Adaptive Cards](#)
  - [Workflow Expressions](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*